

Infrastructure Fundamentals

Deck 1: From Application to Infrastructure

A foundational series for sustain engineers — compute, OS, networking, storage, and data centers

SERIES: 1 OF 3

DECK 1 OF 3

OPENING HOOK

Your code works.

Is your system ready for users?

Running on your laptop is not the same as running in production. This deck builds the mental model between the two.

WHAT YOU ALREADY KNOW

You Know How to Write Application Code

You write classes and methods

Java, Spring, React — logic lives in source files

You run it locally

`mvn spring-boot:run` or `npm run dev` on your machine

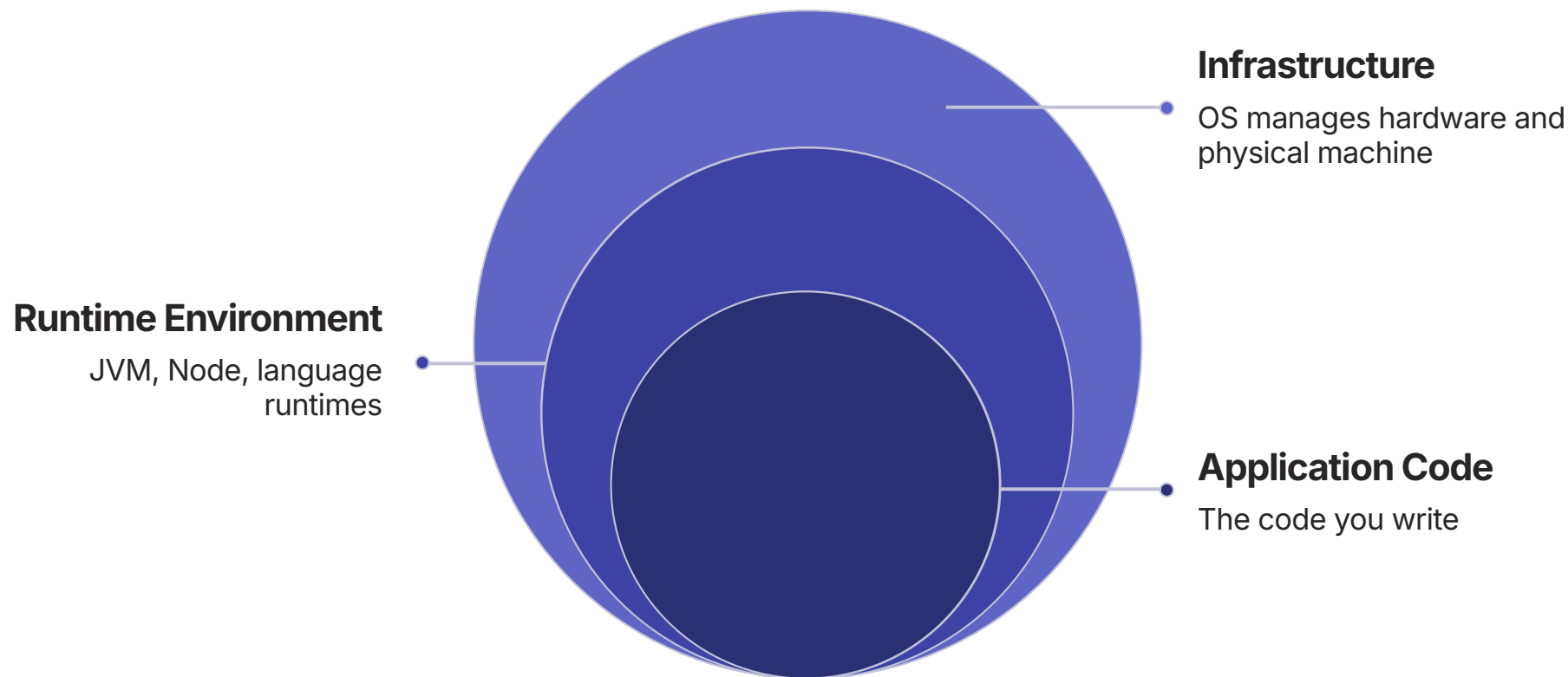
It works on your laptop

Browser opens, data appears, tests pass

This is a great start. But a real production system is much more than source code running on a developer machine.

- ❏ The gap between "works on my laptop" and "works for 10,000 users" is infrastructure.

Application Code Is Only One Layer



Every layer depends on the one beneath it. Infrastructure is everything below your application code.

KEY QUESTION

What does an application need in order to run?

Before we look at servers and networks, let's identify the fundamental resources every running program requires.

COMPUTE

Compute = CPU + Memory + Execution Environment



CPU

Executes instructions. The "brain" of the machine.



RAM

Holds active data and program state while running.



Execution Environment

Runtime that translates your code into CPU instructions (JVM, Node runtime, etc.)

Together, these three things form **compute** — the raw capability to run a program.

Inside a Physical Computer



CPU

Central Processing Unit — executes all program instructions



RAM

Random Access Memory — fast, volatile working memory



Storage

SSD or HDD — persistent data that survives power loss



NIC

Network Interface Card — connects the machine to a network

The CPU Executes Instructions

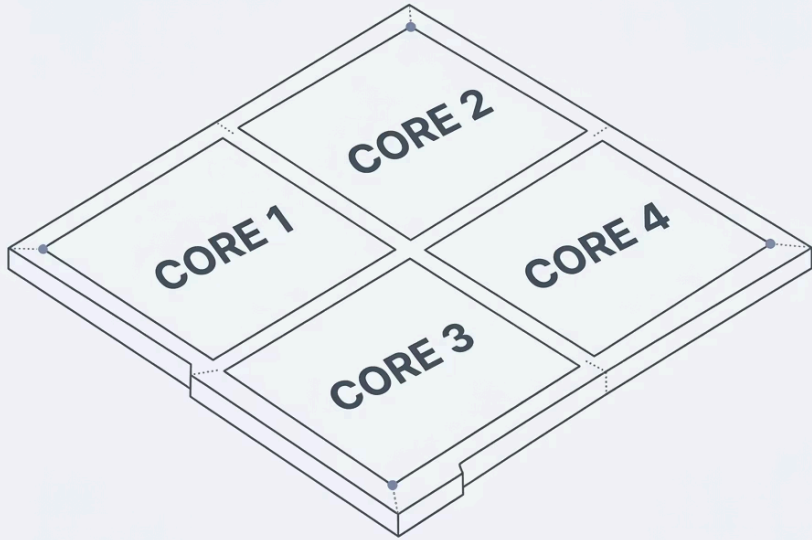


What the CPU actually does

- **Fetch** — retrieve the next instruction from memory
- **Decode** — interpret what the instruction means
- **Execute** — perform the operation (add, compare, store...)

Your Java method calls ultimately become sequences of these low-level instructions.

One CPU, Multiple Cores



Why this matters

- Each core can execute instructions independently
- More cores → more parallelism for multi-threaded applications
- One application process can use multiple cores

❏ A 4-core CPU is not four separate CPUs — it is one chip with four execution units.

RAM

RAM — Active Working Memory

Fast access

CPU can read/write RAM in nanoseconds — far faster than disk

Volatile

Contents are lost when the process stops or power is cut

Limited capacity

Servers have GBs of RAM; running too many apps causes contention

RAM holds everything the running program needs *right now*: variables, objects, call stacks, open file buffers.

Persistent Storage — Data That Survives

SSD

Solid-State Drive — fast, no moving parts, moderate cost

HDD

Hard Disk Drive — slower, spinning platters, lower cost per GB

Key property: non-volatile

Unlike RAM, storage retains data after a process exits or a machine reboots.

- Application binaries live on disk
- Log files are written to disk
- Database files reside on persistent storage

COMPARISON

CPU vs RAM vs Storage

Property	CPU	RAM	Storage
Purpose	Execute instructions	Hold active data	Persist data long-term
Speed	GHz (very fast)	~nanoseconds	~milliseconds (slow)
Volatile?	N/A	Yes — lost on restart	No — survives restart
Capacity unit	Cores / GHz	GB of RAM	GB / TB of disk

MTS Needs All Three Resources



CPU

Spring Boot executes business logic — transfer validation, calculations — on the CPU



RAM

JVM heap holds live objects, HTTP session state, and Spring context in memory



Storage

Dev: H2 in-memory (no disk) |
Production: MySQL on persistent disk storage

📌 H2 stores data only in RAM — it disappears on restart. MySQL writes to disk — data survives.

KEY QUESTION

Can application code talk directly to hardware?

Your Java code doesn't open files by spinning a disk — something in between manages that. What is it?

The OS — Hardware's Manager



CPU Scheduling

Allocates CPU time across all running processes



Memory Management

Controls which process gets which RAM region



File System

Abstracts storage as files and directories

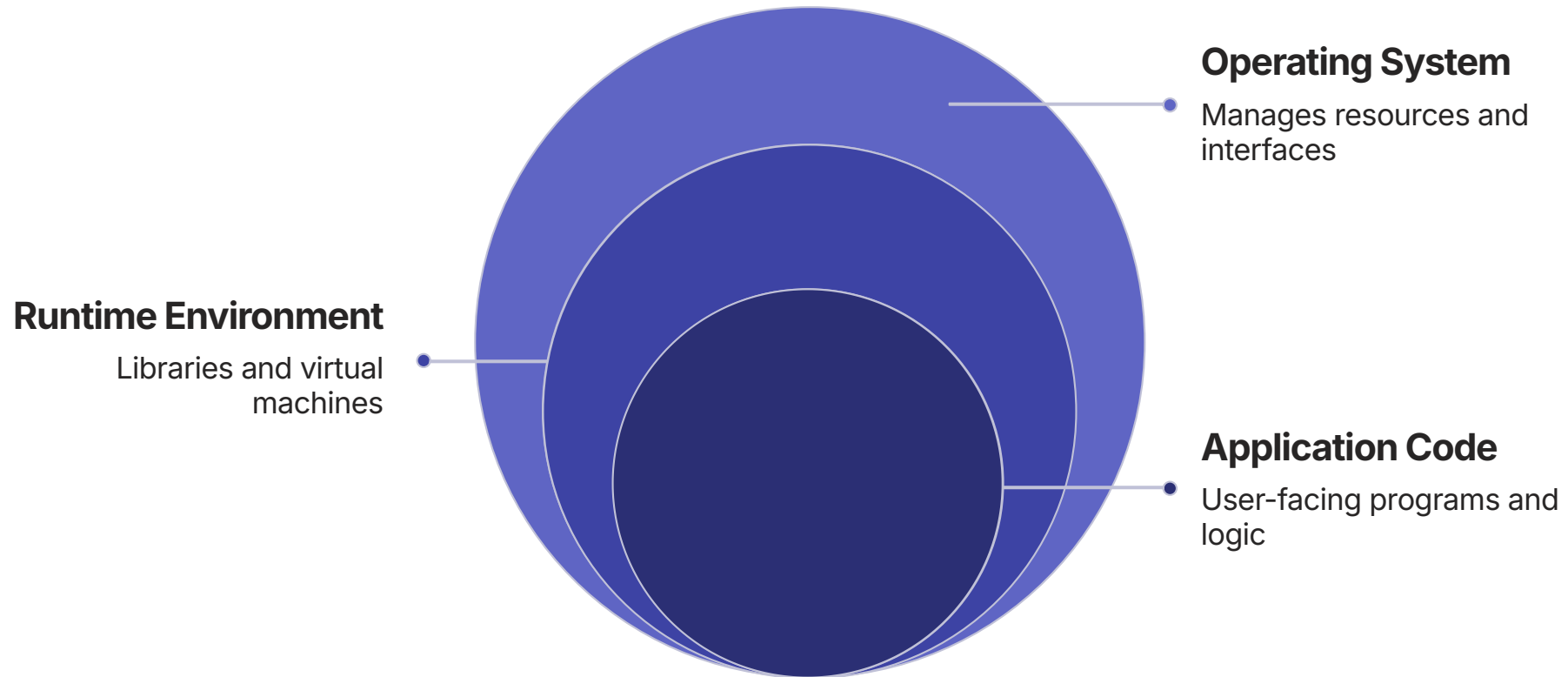


Network & Devices

Manages NIC drivers, sockets, and I/O devices

Applications never talk to hardware directly. Every request goes through the OS.

The Layered Mental Model



Each layer provides services to the layer above. Code never bypasses the OS to touch hardware.

MTS Lives in This Stack

1

Hardware

Physical server: CPU, RAM, disk, NIC

2

Operating System

Linux — manages all hardware resources

3

JVM (Java Runtime)

Java 17 — translates bytecode to CPU instructions

4

Spring Boot Application

MTS backend — your business logic runs here

What Is a Process?

A running program + its allocated resources

- Allocated CPU time by the OS scheduler
- Own region of RAM (virtual address space)
- Open file handles, network sockets
- A unique Process ID (PID)

Source code vs process

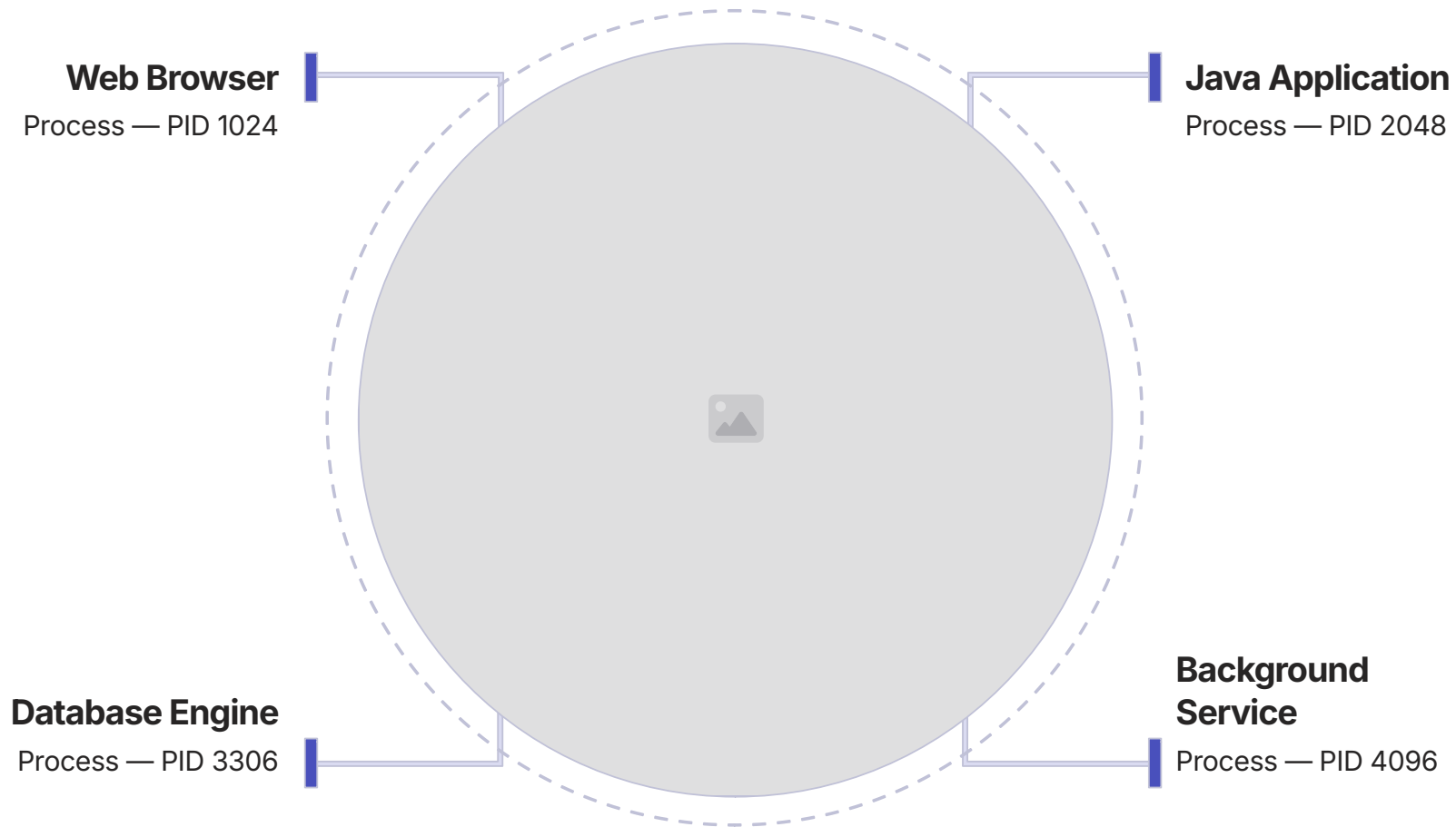
Source code

A file on disk — inert, not running

Process

The compiled program loaded into RAM and actively executing

One Host, Many Processes



The OS runs all processes simultaneously, sharing the same physical CPU and RAM. Each process thinks it owns the machine.

Processes Are Isolated From Each Other

Virtual address space

Each process sees its own memory map — it cannot directly read another process's memory

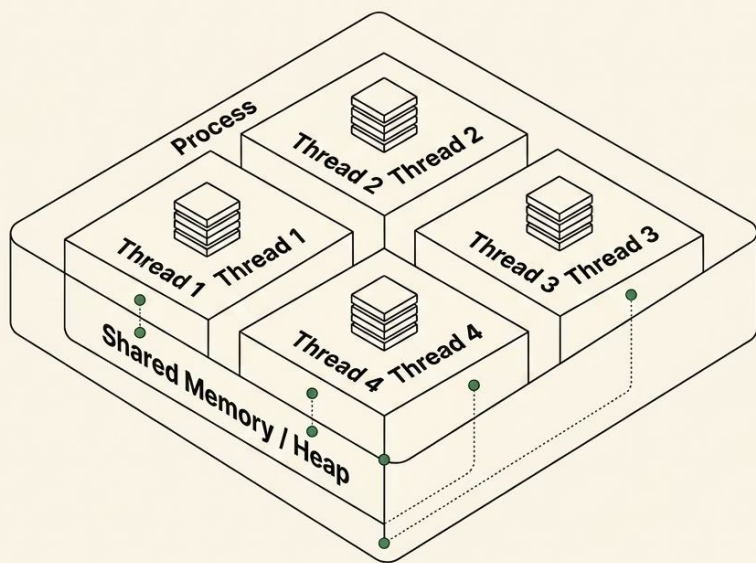
OS enforcement

The OS kernel enforces boundaries; a crashing process does not corrupt its neighbors

Resource accounting

CPU and memory quotas are tracked per-process — runaway apps cannot starve the whole machine

Process Contains Threads — Quick Recap



What you already know from Java

- A process starts with one thread
- Threads share the process heap but have individual stacks
- Spring Boot uses a thread pool to handle concurrent HTTP requests

📄 This slide bridges your Java knowledge. The infrastructure course focuses on processes, not thread internals.

KEY QUESTION

How does another machine reach this application?

The process is running. The OS manages it. But how does a request from another computer find its way in?

Network Interface Card — The Host's Door to the Network



What the NIC does

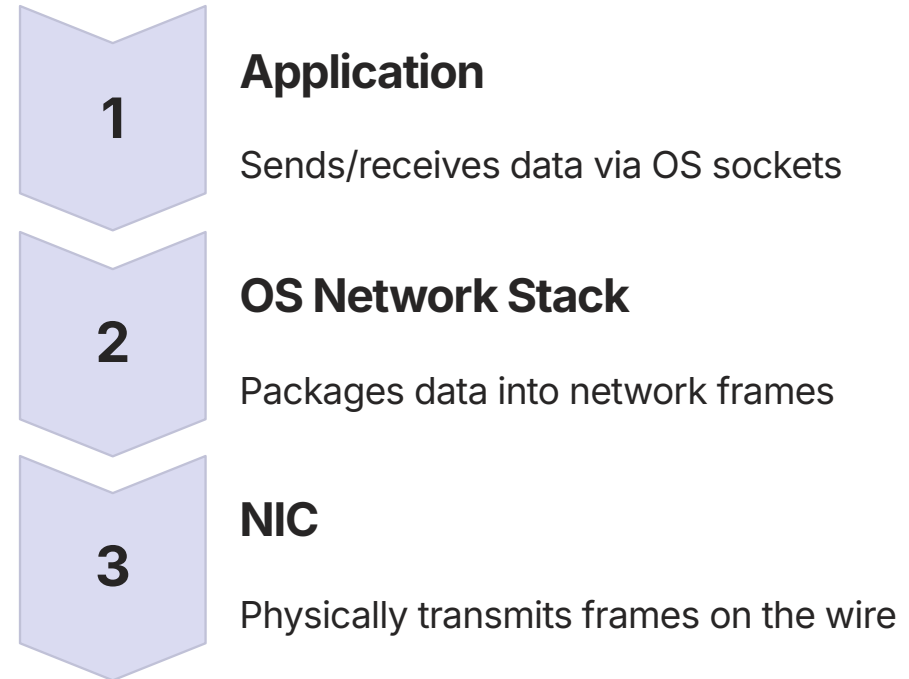
Converts electrical/optical/wireless signals into data the OS can process — and back again



Physical identity

Each NIC has a unique hardware (MAC) address burned in at manufacture

Where it fits in the stack



IP Address — Network-Layer Identity

192.168.1.42

A numeric label assigned to a network interface. Routers use IP addresses to forward packets across networks.

The street-address analogy

Think of an IP address like a postal address for a building. It tells the network *where* to deliver data — but not which service inside to reach. (Ports handle that — coming next.)

❏ IP identifies a **host interface**, not a specific application.

IPv4

Four octets, e.g. 10.0.0.5 — most common today

IPv6

Longer hex format — larger address space for future scale

Private vs Public IP — At a Glance



Private IP

Used inside a network (home, office, data center). Not directly reachable from the public internet.

Examples: 192.168.x.x, 10.x.x.x, 172.16–31.x.x



Public IP

Globally routable on the internet. Assigned by an ISP or cloud provider. Users on the internet reach a server through its public IP.

A server in a data center typically has a private IP internally and a public IP exposed to the internet. We'll revisit this in Deck 2.

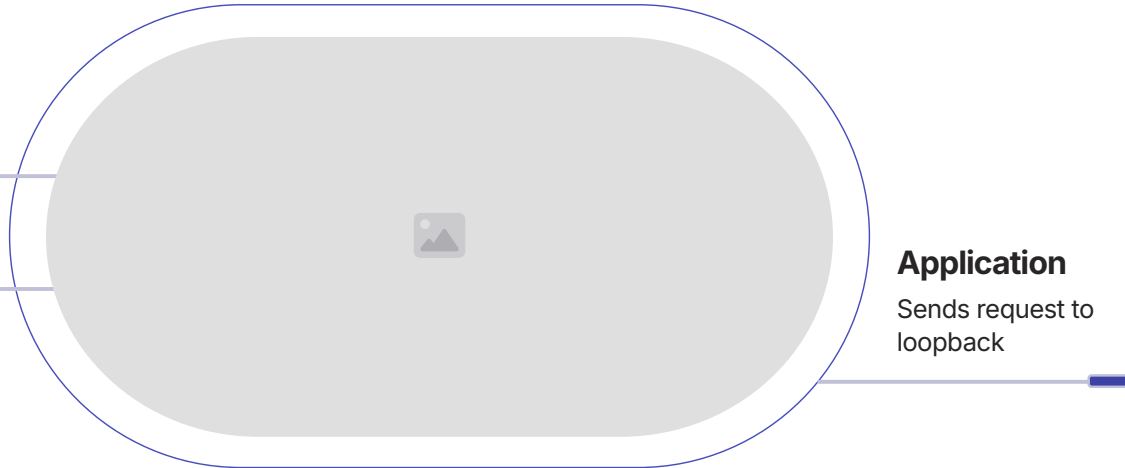
localhost / 127.0.0.1 — This Same Machine

Your Machine

Host boundary
containing local
processes

127.0.0.1

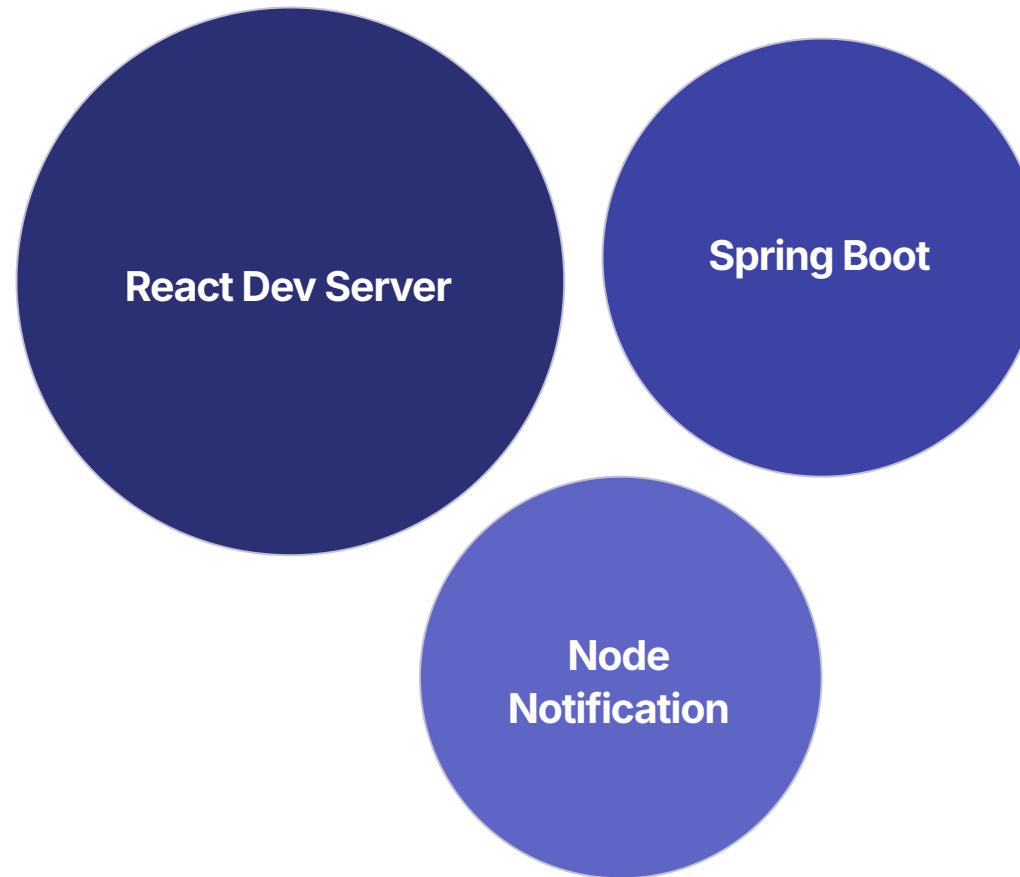
Loopback — never
leaves the host



The loopback interface

- A virtual NIC that the OS creates automatically
- Traffic to 127.0.0.1 stays inside the same machine
- Never goes over a physical network
- localhost is a hostname that resolves to 127.0.0.1

MTS Development: Three Services, One Machine



All three MTS services communicate over loopback in development — no network traffic leaves the machine.

KEY QUESTION

**One machine can run many apps.
How do we reach the right one?**

IP gets packets to the host. But which of the many running processes should receive the traffic?

Ports — Selecting the Right Service

A port is a number (0–65535)

The OS uses the port number to route an incoming network packet to the correct process.

Analogy

If an IP address is the building's street address, a port is the apartment number. The same building (host), different residents (services).

- ❏ One process listens on one port. Two processes cannot share the same port on the same IP simultaneously.

Well-known ports

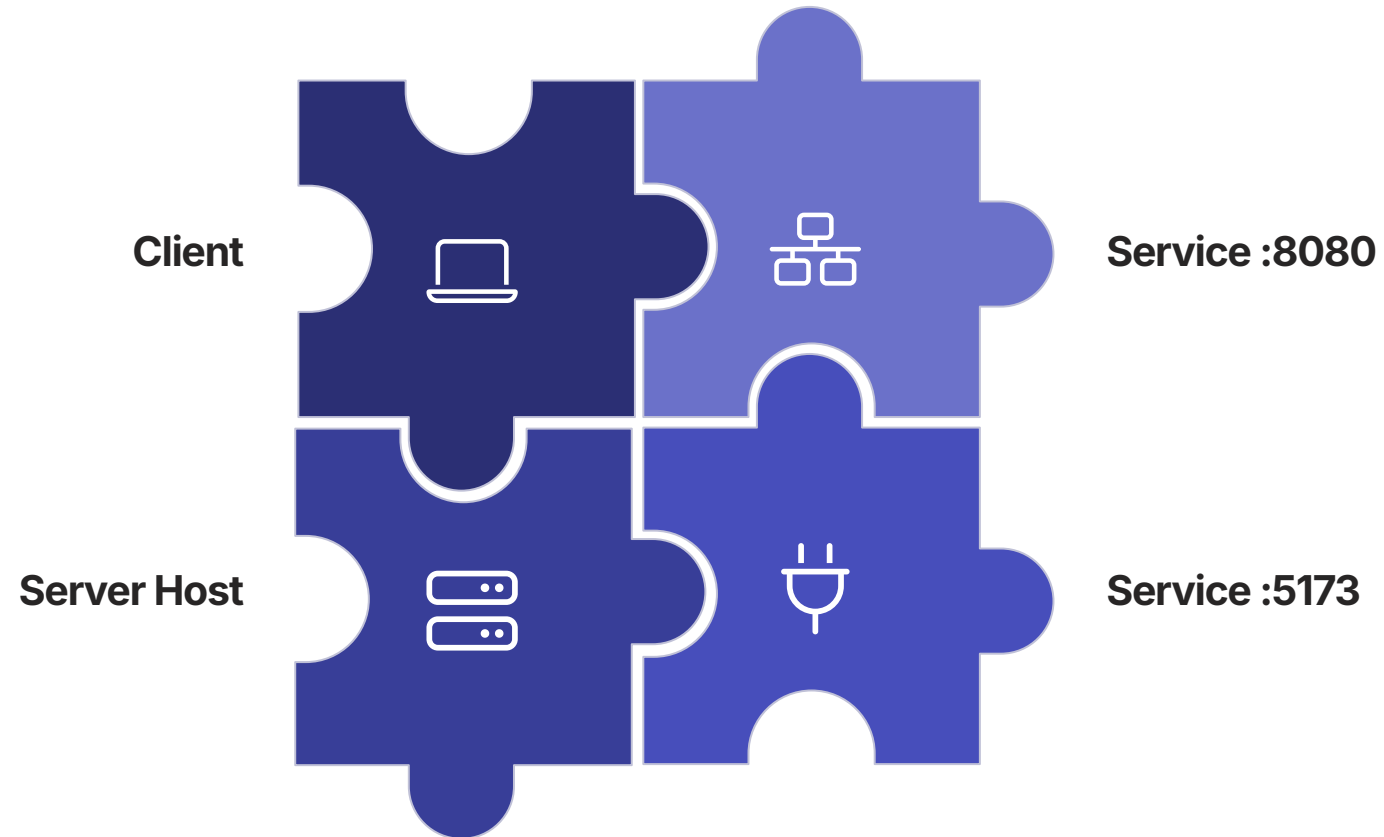
0–1023 — reserved (HTTP: 80, HTTPS: 443, SSH: 22)

Registered / application ports

1024–49151 — commonly used by apps (e.g. 8080, 3000, 5173)

IP + PORT

IP + Port Together Identify a Service



The combination `IP:Port` (called a socket address) uniquely identifies one service endpoint on the network.

MTS Port Map

Service	Runtime	Port
React Frontend (Vite dev server)	Node.js / Vite	5173
Spring Boot Backend API	Java 17 / JVM	8080
Node Notification Service	Node.js / Express	3000

📄 In development, Vite proxies `/api/*` requests to `localhost:8080` so the browser never calls Spring Boot directly on a different port.

Protocol — Agreed Rules for Communication

1**HTTP / HTTPS**

Application protocol — web requests and responses

2**TCP**

Transport protocol — reliable, ordered delivery of bytes

3**IP**

Network protocol — routes packets between hosts

Why protocols matter

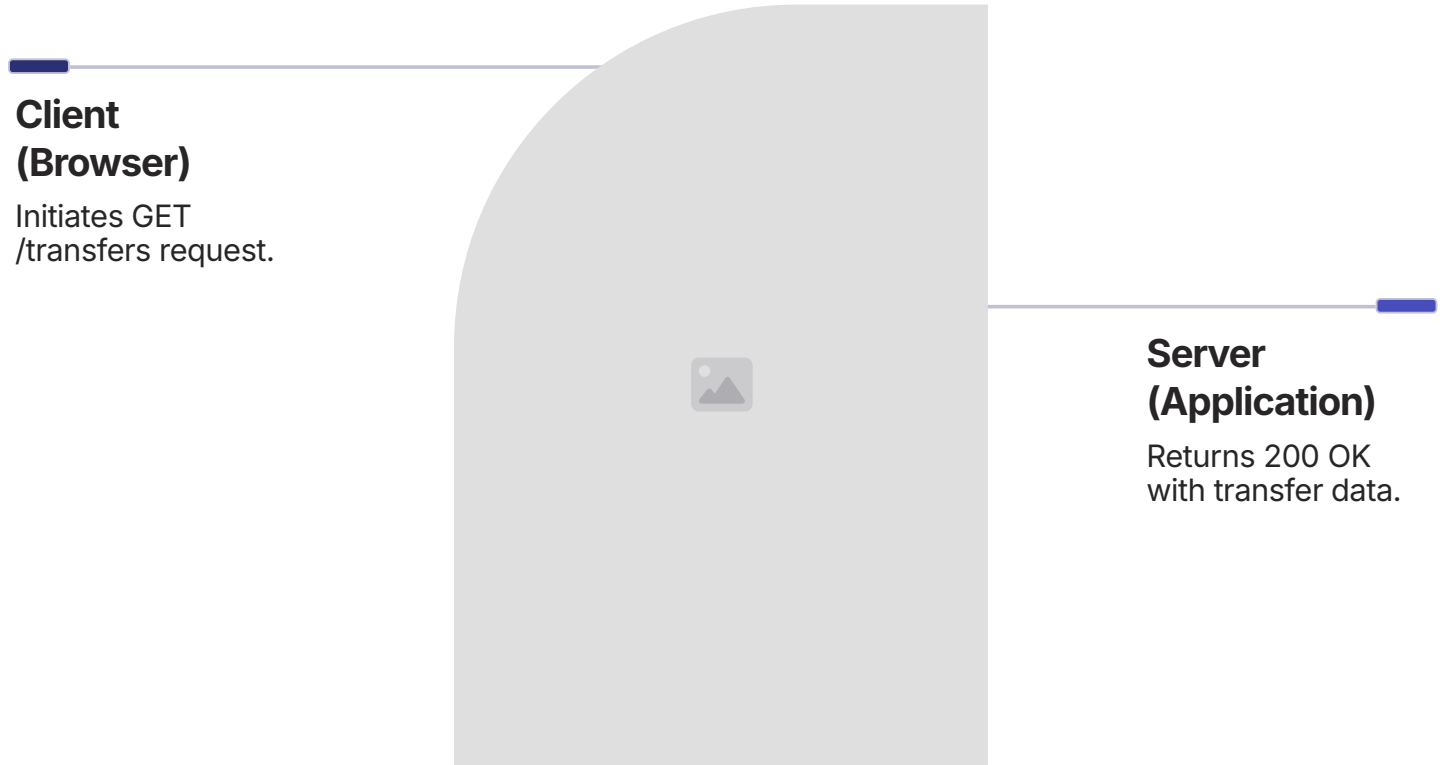
Without agreed rules, two machines cannot parse each other's messages. Protocols define:

- Message format (what bytes mean what)
- Connection setup and teardown
- Error handling and retries

HTTP rides on top of TCP. TCP rides on top of IP. Each layer adds a service.

HTTP

HTTP Request / Response



Client (Browser)

Initiates GET
/transfers request.

Server (Application)

Returns 200 OK
with transfer data.

Request contains

Method (GET/POST/...), path, headers, optional body

Response contains

Status code (200, 404, 500...), headers, response body

HTTPS

HTTPS — HTTP Protected with TLS



Confidentiality

Traffic is encrypted — no eavesdropper on the network can read it



Integrity

Data cannot be tampered with in transit without detection



Authentication

TLS certificate proves the server is who it claims to be

TLS is a handshake layer

Before HTTP data flows, client and server perform a TLS handshake to agree on encryption keys. After that, ordinary HTTP runs inside the encrypted tunnel.



Production services **must** use HTTPS. HTTP sends credentials and data in plain text.

URL Anatomy

Left URL Segments

Scheme and Host shown left-to-right.



Right URL Segments

Port and Path labeled clearly below.

Scheme

`https` — protocol to use

Host

Domain name or IP of the server

Port

Optional — defaults to 80 (HTTP) or 443 (HTTPS)

Path

Resource or endpoint on the server

MTS: Frontend Calls Spring Boot via REST/JSON

Request

React sends `POST /api/transfers` with a JSON body containing transfer details

Development proxy

Vite intercepts `/api/*` and forwards to `localhost:8080` — the browser only sees one origin

Why REST + JSON?

REST uses standard HTTP verbs and URLs. JSON is a human-readable, lightweight data format. Together they make cross-service communication predictable and easy to debug.

Response

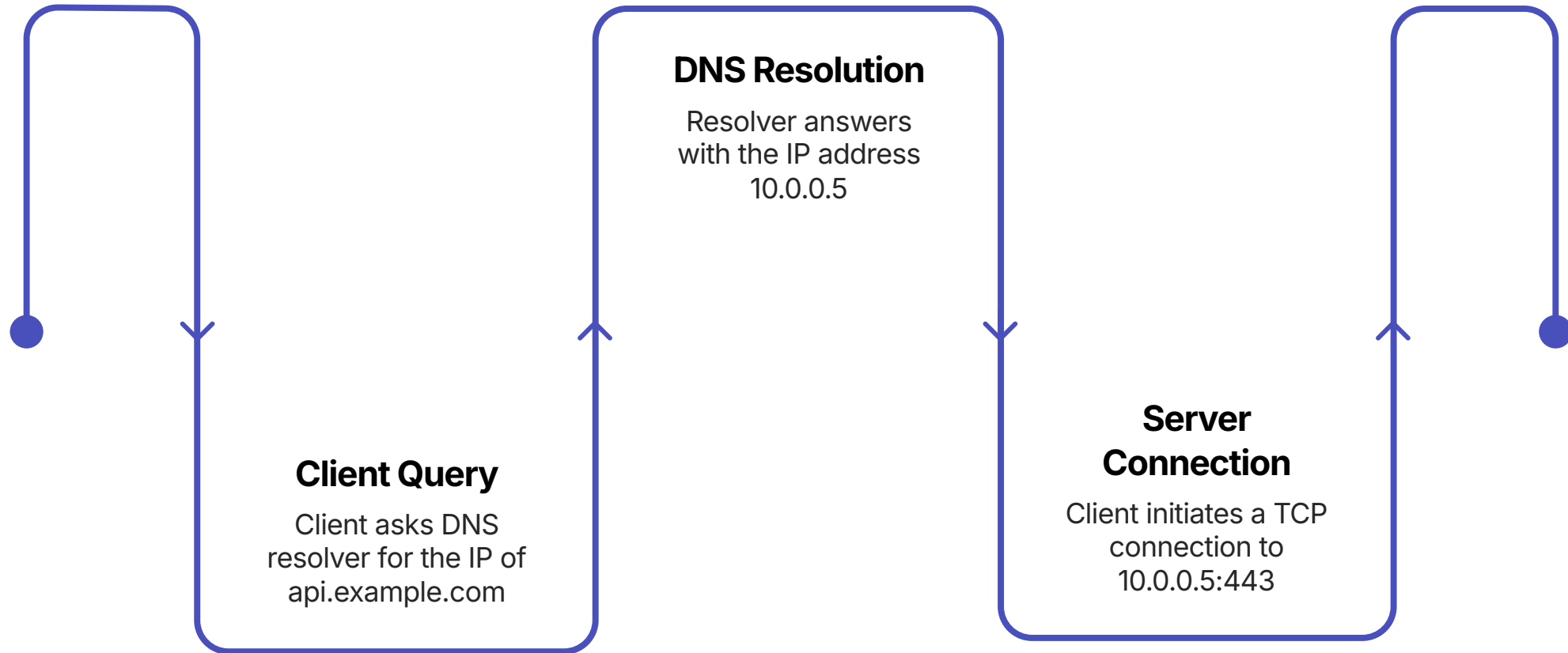
Spring Boot returns JSON — transfer result, status, confirmation

KEY QUESTION

Users remember names, not IP addresses.

Nobody types 185.199.108.153 into their browser. How does a human-readable name get translated to an IP?

DNS — Name to IP Resolution



DNS resolves a name to an IP. After resolution, the *client* opens a direct connection to the server. DNS does not carry application data.

DNS Resolution — Simplified Sequence

01

Check local cache

Browser and OS cache recent DNS answers. If found, skip the rest.

03

Resolver looks up the answer

Resolver queries authoritative servers if needed and returns the IP address

02

Query the resolver

OS asks the configured DNS resolver (often provided by ISP or corporate network)

04

Cache & connect

Result is cached for the TTL period; client connects directly to the IP

DNS

Domain Name vs IP Address



Domain Name

api.example.com

Human-readable and stable. Doesn't change even when infrastructure moves to a new IP.



IP Address

185.199.108.153

Machine-readable network address. Can change when servers are replaced, migrated, or scaled.

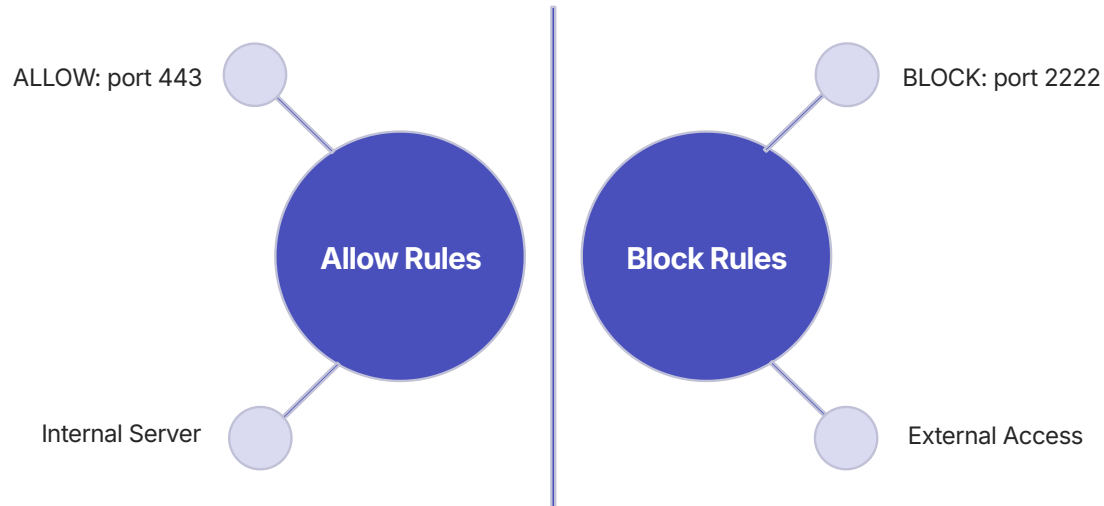
DNS is the indirection layer that lets names remain stable while the underlying IP changes — critical for production systems.

KEY QUESTION

Should every network request be allowed into the server?

A server exposed to the internet receives traffic from everywhere — legitimate users and attackers alike. What decides what gets through?

Firewall — Traffic Gatekeeper



How a firewall works

A firewall evaluates each packet against a set of rules:

- **Source IP** — where is it coming from?
- **Destination port** — which service is being targeted?
- **Protocol** — TCP, UDP?
- **Direction** — inbound or outbound?

If a rule matches → allow or deny. If no rule matches → default policy applies (usually deny).

Inbound vs Outbound Traffic

Inbound

Traffic arriving *at* the server from the outside world.

Example: a user's browser connecting to port 443.

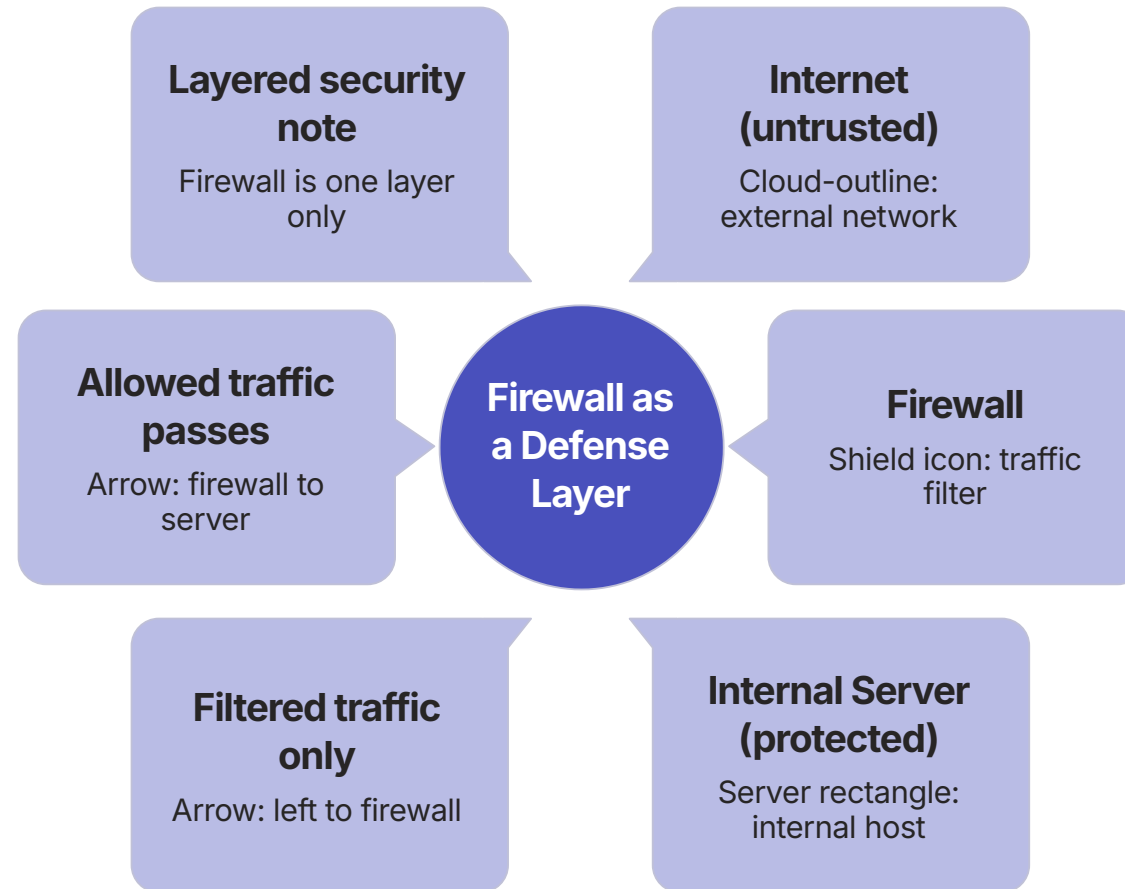
Firewalls typically restrict inbound traffic to only the required ports.

Outbound

Traffic leaving the server toward the outside. Example: the app querying an external payment API.

Outbound rules prevent a compromised server from "phoning home" to attackers.

The Security Boundary



⚠ A firewall is a necessary layer of defense — but not sufficient alone. Application security, authentication, and encryption are also required.

Conceptual Production Path for MTS



In a real production deployment, user traffic would pass through DNS resolution and a firewall before reaching the Spring Boot endpoint. This is not yet in the MTS repository — it is the target architecture.

KEY QUESTION

Where does application data live after the process stops?

RAM is volatile. When Spring Boot restarts, every object in the JVM heap is gone. How do we persist data across restarts?

The Role of a Database



Durable persistence

Data survives process restarts, crashes, and power cycles



Structured query

SQL lets applications find, filter, and aggregate records efficiently



Transactions

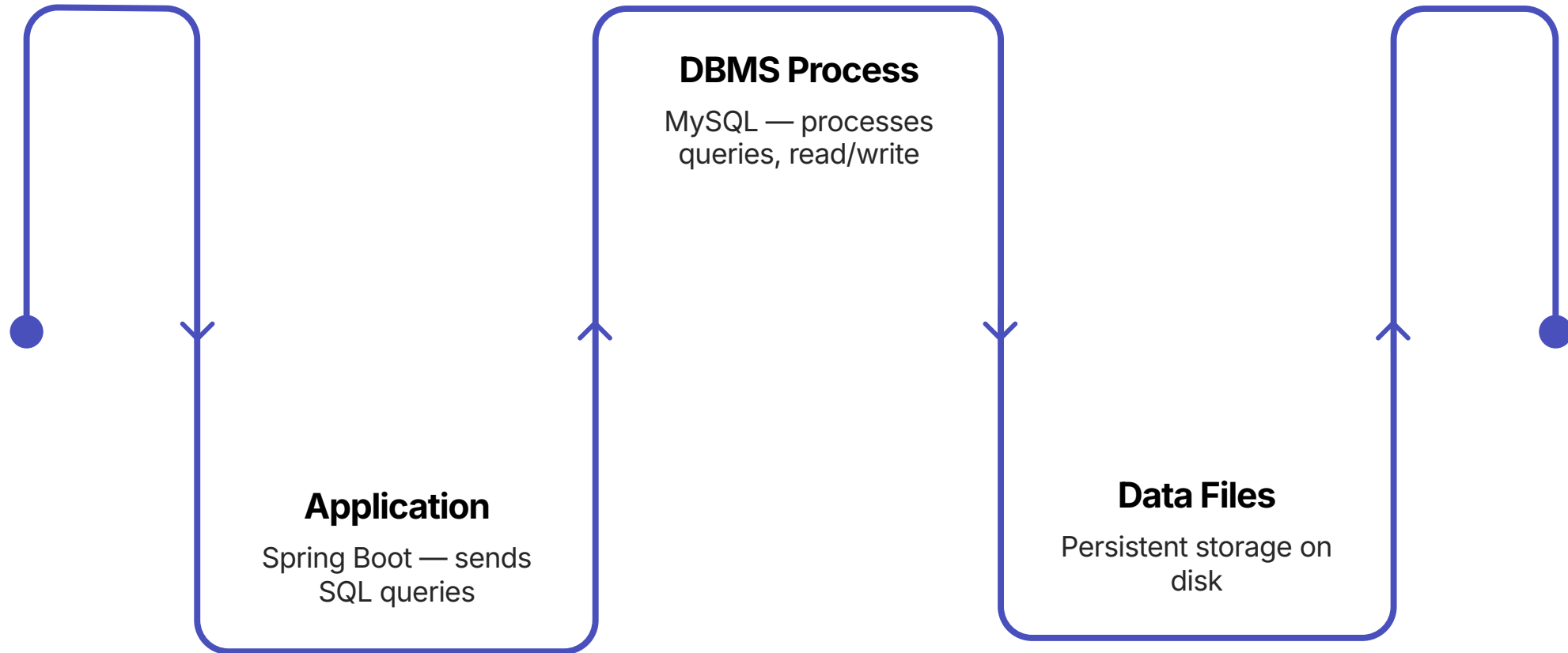
ACID guarantees ensure partial updates don't leave data in an inconsistent state

Key distinction

A database is **software** (MySQL, PostgreSQL) that manages data on top of **storage hardware** (a disk). The two are separate concerns.

- ❏ The database process handles queries, indexing, locking, and caching. The storage hardware just holds bits.

DBMS, Data Files, and Storage — Three Distinct Things



The application talks to the DBMS. The DBMS manages data files on disk. The application never reads raw data files directly.

MTS: H2 for Dev, MySQL for Production



Development — H2 In-Memory

Database runs inside the JVM. No separate process. No disk files. Data is lost on every restart.

Spring profile: `default` | Great for fast iteration and unit tests.



Production — MySQL

Separate database process. Data written to persistent disk storage. Survives Spring Boot restarts.

Spring profile: `production` | Required for real user data.



Using H2 in production means all data is lost on every restart. Always use MySQL (or equivalent) in production.

Three Infrastructure Storage Abstractions



Block Storage

Raw disk blocks — fast, low-level. Databases and OS filesystems are built on top of it.



File Storage

Familiar files and folders. What your OS presents to applications — built on top of block storage.



Object Storage

Flat namespace of named blobs (images, documents, backups). Accessed via HTTP API. Highly scalable.

Most production database deployments use block storage for maximum performance and control.

Local vs Network-Attached Storage



Local Storage

Disk physically inside or directly attached to the server.

- Lowest latency — direct hardware bus
- Tied to one machine — if the server fails, disk is unavailable
- Good for high-performance, single-node databases



Network-Attached Storage

Disk accessed over a network from a separate storage system.

- Shared by multiple servers — enables flexibility
- Higher latency than local (network round-trip added)
- Storage survives server failure — data still accessible

The I/O Path — From Application to Disk



I/O latency is orders of magnitude slower than CPU/RAM operations. Slow storage is one of the most common production bottlenecks.

KEY QUESTION

Where do production servers physically live?

We've built the logical stack. Now — where is the physical machine? Who powers it, cools it, and connects it to the internet?

The Data Center — A Controlled Environment for Servers



Racks & Servers

Servers are mounted in standardized metal racks — dense, organized, easy to cable



Network Infrastructure

Switches, routers, and cabling connect every server to each other and to the internet



Power Systems

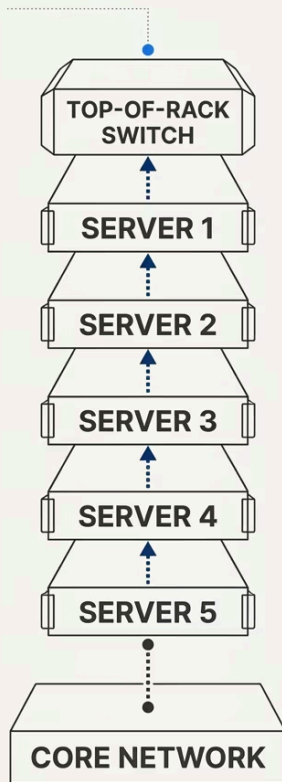
Redundant utility feeds, UPS units, and diesel generators ensure uptime during outages



Cooling

Precision air conditioning maintains temperature and humidity — heat kills hardware

Rack, Server, and Network Layout



Key concepts

- A **rack** holds ~40–48 rack-unit servers in a standard 42U cabinet
- A **top-of-rack switch** connects all servers in the rack to each other and to the data center network
- Servers communicate at high speed (10/25/100 Gbps) within a rack
- Traffic to the internet exits through core routers at the data center edge

Why Cooling Matters

Heat is a by-product of compute

Every CPU, RAM module, and NIC generates heat as it processes instructions

Thermal throttling

CPUs reduce their clock speed when too hot — your application slows down before hardware fails

Hardware failure

Sustained overheating causes permanent damage to chips, capacitors, and storage

How data centers manage heat

- Raised floors with cold air supply vents below racks
- Hot aisle / cold aisle layout to direct airflow
- Precision computer room air conditioning (CRAC) units
- Temperature and humidity sensors with automated alerts

❏ Power Usage Effectiveness (PUE) measures how much data center power goes to compute vs cooling — a key efficiency metric.

The Foundational Production Stack



This is the complete single-server on-premises production baseline we've built across this deck. Every component has been introduced and explained.

MTS CONNECTION

CURRENT CODE + CONCEPTUAL PRODUCTION

MTS Mapped onto the Production Stack



The notification service is a side dependency of Spring Boot — not a database component. Transfer succeeds even if notifications fail.

We Have One Stack. Now the Hard Questions Begin.

What happens next?



Traffic grows

One server runs out of CPU and RAM.
How do we scale?



One server isn't enough

We need multiple servers. How do we
distribute requests across them?



Infrastructure on demand

Can we provision and discard servers
programmatically instead of buying
hardware?